

PROE_WGR-V: Entwurf und Implementierung eines RISC-V Prozessorsystems

Projektarbeit E

**Fachhochschule Kiel
Fachbereich Informatik und Elektrotechnik
Sommersemester 2025**

Tobias Kling

Philip Mohr

Zusammenfassung

Dieses Dokument beschreibt das Projekt PROE_WGR-V, das im Rahmen der Projektarbeit E an der Fachhochschule Kiel im Sommersemester 2025 von Tobias Kling und Philip Mohr umgesetzt wurde. Ziel des Projekts war die Entwicklung eines einfachen, RISC-V-kompatiblen Prozessorkerns (RV32I/E) sowie einiger Peripheriemodule in Verilog. Zusätzlich wurden grundlegende Software-Tools, Testmöglichkeiten und eine Hardware Abstraction Layer (HAL) in C erstellt. Das System läuft auf FPGAs und kann grundsätzlich auch für ASICs genutzt werden. Diese Arbeit dokumentiert das Gesamtkonzept, die wichtigsten Komponenten und gibt Hinweise zur Nutzung und möglichen Weiterentwicklung.

Inhaltsverzeichnis

Inhaltsverzeichnis	i
Abbildungsverzeichnis	iii
1 Einleitung	1
1.1 Motivation und Zielsetzung	1
1.2 Projektübersicht	2
1.3 Struktur des Dokuments	2
2 Gesamtkonzept und Systemarchitektur	4
2.1 RISC-V Grundlagen	4
2.2 Architektur des WGR-V Prozessors	4
2.3 Speicherorganisation	6
2.3.1 Adressraumaufteilung	6
2.3.2 RAM-Implementierung	6
2.3.3 Systemtakt und Zeitgeber	6
2.4 Peripherieanbindung	7
2.4.1 Adressdekodierung	7
2.4.2 Datenfluss und Steuersignale	7
3 Hardwareimplementierung in Verilog	9
3.1 Projektstruktur /rtl	9
3.1.1 CPU-Kern (cpu.v)	9
3.1.2 ALU (alu.v)	9
3.1.3 Register File (register_file.v)	10
3.1.4 Top-Level Modul (wgr_v_max.v)	10
3.2 Peripheriemodule (/rtl/peripherals)	10
3.2.1 Peripheral Bus (peripheral_bus.v)	10
3.2.2 UART (uart.v)	10
3.2.3 SPI (spi.v)	11
3.2.4 PWM Timer (pwm_timer.v)	11

3.2.5	Debug Module (debug_module.v)	11
3.2.6	System Timer (system_timer.v)	11
3.2.7	FIFO (fifo.v)	11
3.2.8	GPIO (gpio.v)	11
3.2.9	WS2812B Controller (ws2812b.v)	11
3.2.10	Sequentieller Dividierer (seq_divider.v)	12
3.2.11	Sequentieller Multiplizierer (seq_multiplifier.v)	12
3.3	FPGA Implementierung (/WGR-V-MAX)	12
4	ASIC Implementierung	13
4.1	Übersicht und Repository	13
4.2	Tools und Prozess	13
4.3	CI/CD Workflow im ASIC Repository	14
4.4	Speicheranbindung im ASIC	14
4.5	Chip-Layout	15
4.6	Enthaltene Peripherie im ASIC	15
5	Software und Firmware (/src)	17
5.1	Übersicht und Dokumentation	17
5.2	Hardware Abstraction Layer (HAL)	17
5.3	Linker und Startup-Code	19
5.4	Beispielprojekte	19
5.5	Funktionale Tests (/src/tests)	20
6	Entwicklungsworkflow und Werkzeuge	23
6.1	Skripte zur Automatisierung (/scripts)	23
6.1.1	Konfigurationsskripte	23
6.1.2	Hex-Konvertierungsskripte	23
6.1.3	Kompilierungs- und Simulationsskripte	25
6.2	Dokumentationsgenerierung	25
7	Anleitung zur Nutzung des Projekts	27
7.1	Voraussetzungen	27
7.2	Kompilieren von Softwareprojekten	28
7.3	Hex-Dateien manuell generieren	29
7.4	Simulation starten	30
7.5	Alles in einem Schritt (build_copy_simulate.bat)	30
8	Zusammenfassung und Ausblick	32
8.1	Erreichte Ziele und Ergebnisse	32
8.2	Mögliche zukünftige Arbeiten	33

Abbildungsverzeichnis

2.1	Vereinfachtes Blockdiagramm der WGR-V Systemarchitektur.	5
2.2	Prinzip der Adressdekodierung: Die Bits [12:8] steuern die <i>Modulwahl</i> (links). Die Bits [7:0] werden als <i>Register-Offset</i> an alle Module verteilt (rechts), aber nur vom ausgewählten Modul (hier: UART) verwendet. . . .	7
4.1	Gerenderte Darstellung des WGR-V ASIC Layouts.	15

Listings

6.1	Aufrufbeispiel für conv_hex.py	24
6.2	Aufrufbeispiel für conv_fram.py	24
7.1	Manueller Aufruf von conv_hex.py	29
7.2	Manueller Aufruf von conv_fram.py	29
7.3	Aufruf von run.bat für die Simulation	30

Kapitel 1

Einleitung

Das Projekt PROE_WGR-V beschäftigt sich mit dem Entwurf und der Umsetzung eines eigenen Prozessorsystems auf Basis der RISC-V Architektur. Die Arbeit entstand im Rahmen der Projektarbeit E an der Fachhochschule Kiel und dokumentiert die Entwicklung eines funktionalen RISC-V Prozessorkerns, der Peripherie und der benötigten Software-Tools. Der gesamte Aufbau ist in Verilog realisiert und kann sowohl auf FPGAs als auch experimentell als ASIC genutzt werden.

1.1 Motivation und Zielsetzung

Hauptziel des Projekts war es, praktische Erfahrungen im digitalen Schaltungsentwurf und mit moderner Prozessorarchitektur zu sammeln. Die offene RISC-V Architektur eignet sich dafür besonders, da sie frei nutzbar ist und keine Lizenzkosten verursacht.

Konkret sollten folgende Punkte umgesetzt werden:

- Entwicklung eines 32-Bit RISC-V CPU-Kerns (RV32I und RV32E) in Verilog.
- Anbindung verschiedener Peripheriemodule wie UART, SPI, Timer, GPIO, einem WS2812B-Controller und einem Hardware-Dividierer.
- Aufbau einer Speicherarchitektur mit On-Chip-RAM (z.B. Altera `altsyncram`) für FPGAs und externem Speicher (SPI-FRAM) für die ASIC-Version.
- Erstellung einer Toolchain und Skripten zur Automatisierung von Kompilierung und Simulation.
- Entwicklung einer HAL in C zur Vereinfachung der Softwareentwicklung und besseren Portierbarkeit.
- Durchführung von Tests zur Überprüfung der Funktionalität und Leistung.

- Erprobung einer ASIC-Implementierung mit dem SkyWater 130nm Prozess und Open-Source Tools.

Das Ziel war ein lauffähiges, gut dokumentiertes RISC-V System, das als Lernplattform dient und später ausgebaut werden kann.

1.2 Projektübersicht

PROE_WGR-V umfasst Hardware, Software und die nötigen Werkzeuge. Der zentrale Baustein ist ein in Verilog geschriebener RISC-V Prozessor (RV32I/E), der über einen selbst entwickelten Bus mit verschiedenen Peripheriemodulen verbunden ist. Dazu gehören Standardmodule wie UART, SPI, PWM-Timer, System-Timer, GPIO und Debug-Schnittstellen. Es gibt auch spezielle Module wie einen WS2812B-LED-Controller und einen Hardware-Dividierer, die teils noch nicht fertiggestellt sind.

Für die Softwareentwicklung steht eine HAL in C zur Verfügung, die den Zugriff auf die Peripherie erleichtert. Beispielprojekte in C und Assembler zeigen die Nutzung des Systems. Verschiedene Tests prüfen die CPU-Funktion und die Performance.

Zur Entwicklung und Nutzung gibt es eine Reihe von Skripten (meist Python und Batch), die das Kompilieren von Programmen, die Konvertierung in passende Formate und die Simulation (z.B. mit QuestaSim) vereinfachen.

Das Projekt ist für zwei Plattformen ausgelegt:

1. **FPGA:** Vor allem für Intel/Altera FPGAs (z.B. MAX 10), mit On-Chip RAM und Nutzung von Quartus.
2. **ASIC:** Eine erste Test-Implementierung für den SkyWater 130nm Prozess, die externen SPI-FRAM nutzt und auf Open-Source Tools setzt.

Alle Projektdaten werden in einem Git-Repository verwaltet. Die Dokumentation wird automatisch mit Doxygen und DoxNet erzeugt.

1.3 Struktur des Dokuments

Das Dokument ist in mehrere Hauptkapitel gegliedert:

- **Kapitel 2:** Überblick über das Gesamtkonzept und die Systemarchitektur des Projekts.
- **Kapitel 3:** Beschreibung der Hardware-Implementierung in Verilog, inklusive CPU-Kern und Peripherie.
- **Kapitel 4:** Besonderheiten der ASIC-Version, eingesetzte Tools und Aufbau.

- **Kapitel 5:** Infos zur Software, wie der HAL, Linker-Skripten und Beispielprojekten.
- **Kapitel 6:** Überblick über die Entwicklungswerkzeuge und genutzte Skripte.
- **Kapitel 7:** Anleitung zur Benutzung des Projekts.
- **Kapitel 8:** Zusammenfassung und Ausblick.

Jedes Kapitel gibt einen kurzen Überblick und erklärt wichtige Details zum besseren Verständnis des Projekts.

Kapitel 2

Gesamtkonzept und Systemarchitektur

Das Ziel von PROE_WGR-V ist ein flexibles RISC-V Prozessorsystem, das sowohl zum Lernen als auch für praktische Einsätze auf FPGAs und als Test-ASIC geeignet ist. In diesem Kapitel werden die grundlegenden Designideen, die Systemarchitektur und das Zusammenspiel der Hauptkomponenten vorgestellt.

2.1 RISC-V Grundlagen

Als Basis dient die RISC-V Instruktionssatzarchitektur (ISA), die offen und frei von Lizenzgebühren ist. RISC-V ist modular aufgebaut und kann je nach Anwendung angepasst werden. Im Projekt lag der Fokus auf den Basis-Integer-Instruktionssätzen:

- **RV32I:** Der Standard-32-Bit-Integer-Instruktionssatz mit den wichtigsten Rechen-, Speicher- und Kontrollbefehlen.
- **RV32E:** Eine abgespeckte Variante mit nur 16 Registern, geeignet für kleine FPGAs oder kompakte ASICs. Die Registeranzahl kann im WGR-V Prozessor leicht umgestellt werden.

Durch die Offenheit und klare Struktur von RISC-V ist die Implementierung gut nachvollziehbar, was das Projekt für Studierende besonders geeignet macht.

2.2 Architektur des WGR-V Prozessors

Der WGR-V Prozessor verbindet einen zentralen CPU-Kern mit Speicher und mehreren Peripheriemodulen. Die Architektur ist klar in verschiedene Funktionsblöcke unterteilt, damit das System modular und gut testbar bleibt.

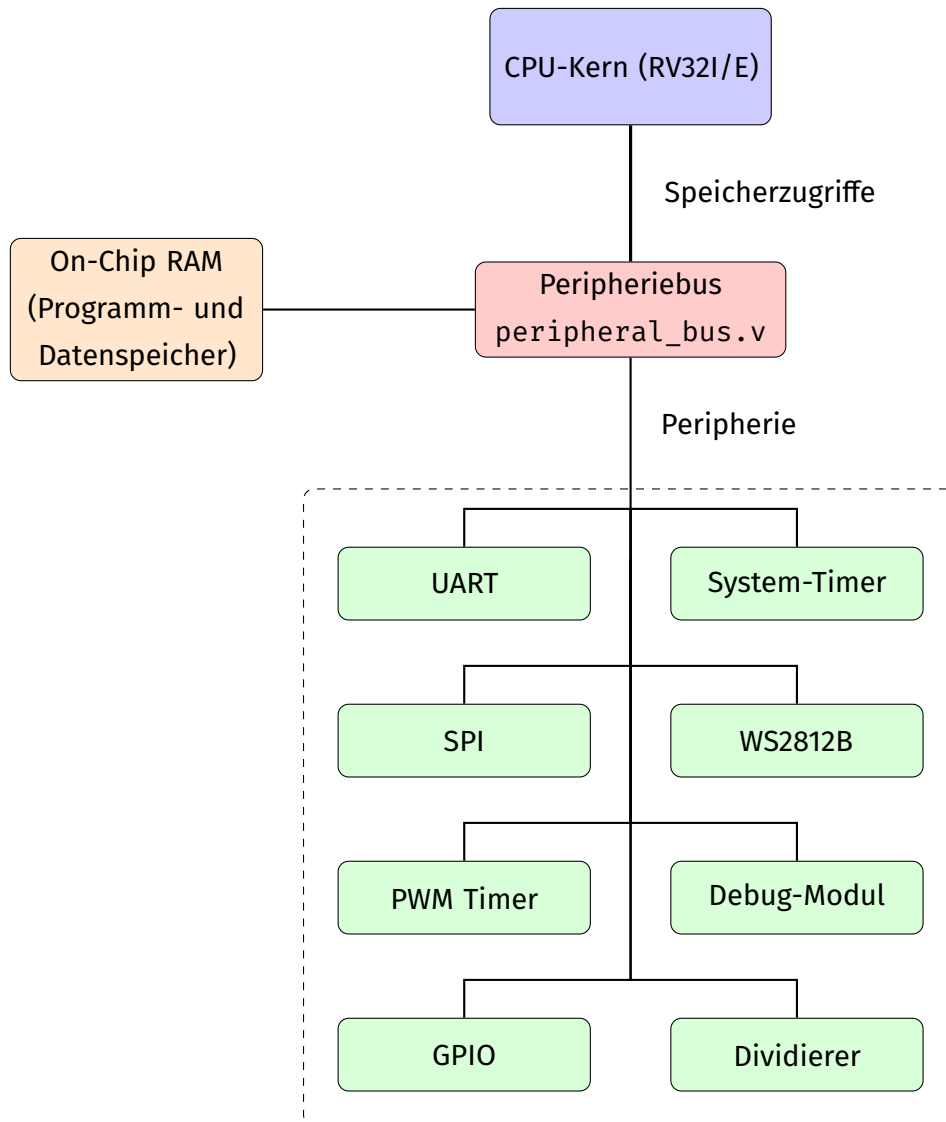


Abbildung 2.1: Vereinfachtes Blockdiagramm der WGR-V Systemarchitektur.

Die wichtigsten Bausteine sind:

- **CPU-Kern (cpu.v):** Führt RISC-V Instruktionen (RV32I/E) aus. Instruktionsarten sind im Code als `localparam` hinterlegt.
- **Speicherinterface:** Verbindet die CPU mit dem Speicher. Über ein `mem_busy` Signal kann die CPU gestoppt werden, falls ein Speicherzugriff länger dauert.
- **Peripheriebus (peripheral_bus.v):** Steuert den Zugriff der CPU auf die Peripheriemodule.
- **Speicher (RAM):** Dient als Programm- und Datenspeicher; die genaue Umsetzung hängt von der Zielplattform ab.
- **Peripheriemodule:** Erweiterungen wie UART, SPI, Timer, GPIO usw., die für verschiedene Aufgaben genutzt werden können.

Durch diese Aufteilung kann Software effizient auf die Hardware zugreifen und Peripherie steuern.

2.3 Speicherorganisation

Im WGR-V System ist der Speicher linear und byteweise adressierbar, wie bei RISC-V üblich. Der Adressraum wird für Programm, Daten und die Register der Peripheriemodule genutzt.

2.3.1 Adressraumaufteilung

Typischerweise ist ein Bereich des Adressraums für den RAM reserviert (z.B. Programm und Daten), andere Bereiche sind für die Peripherie vorgesehen. Bei der Initialisierung des Speichers wird ein sogenannter `shift_amount` (z.B. `0x4000`) genutzt: Die Software wird eigentlich ab Adresse `0x00000000` gelinkt, landet physisch aber ab dem Wert des Shifts im RAM. Die Bereiche darunter stehen dann für Peripheriegeräte zur Verfügung.

Zum Beispiel: Greift die CPU auf `0x4008` zu und der `shift_amount` ist `0x4000`, dann wird intern auf Adresse `0x0008` im RAM zugegriffen.

2.3.2 RAM-Implementierung

Die RAM-Umsetzung hängt von der Zielplattform ab:

- **FPGA-Version (/WGR-V-MAX):** Es wird ein synchroner RAM-Block (z.B. `altsyncram` von Intel/Altera) verwendet. Dieser IP-Kern wird im Quartus-Projekt eingebunden und mit einer `.hex`-Datei (z.B. `wgr_flat.hex`) vorinitialisiert. Im Beispiel sind das 32 KiB RAM (8192 x 32 Bit).
- **ASIC-Version:** Hier kommt ein externer SPI-FRAM zum Einsatz, der über ein spezielles SPI-Interface (u.a. `fram_ram.v`, `fram_spi.v`, `mb85rs64v.v`) angebunden ist. Die CPU verwendet ihn wie normalen RAM.

2.3.3 Systemtakt und Zeitgeber

Die Datei `defines.v` legt die CPU-Frequenz (`CLK_FREQ`) fest. Diese Angabe ist wichtig für Module wie UART (Baudrate) und System-Timer (korrekte Zeitmessung). Eine falsche Einstellung kann zu Fehlern führen. Der System-Timer stellt Zähler für Milli- und Mikrosekunden bereit, die über Speicheradressen ausgelesen werden können.

2.4 Peripherieanbindung

Die Peripheriemodule sind über das Modul `peripheral_bus.v` an die CPU angebunden. Dieses Modul übernimmt die Adressauswertung und leitet Zugriffe entweder an den RAM oder das passende Peripheriemodul weiter.

2.4.1 Adressdekodierung

Der Bus nutzt bestimmte Adressbits, um das Zielmodul auszuwählen: `address[12:8]` wählt das Peripheriemodul, `address[7:0]` bestimmt das Register innerhalb des Moduls.

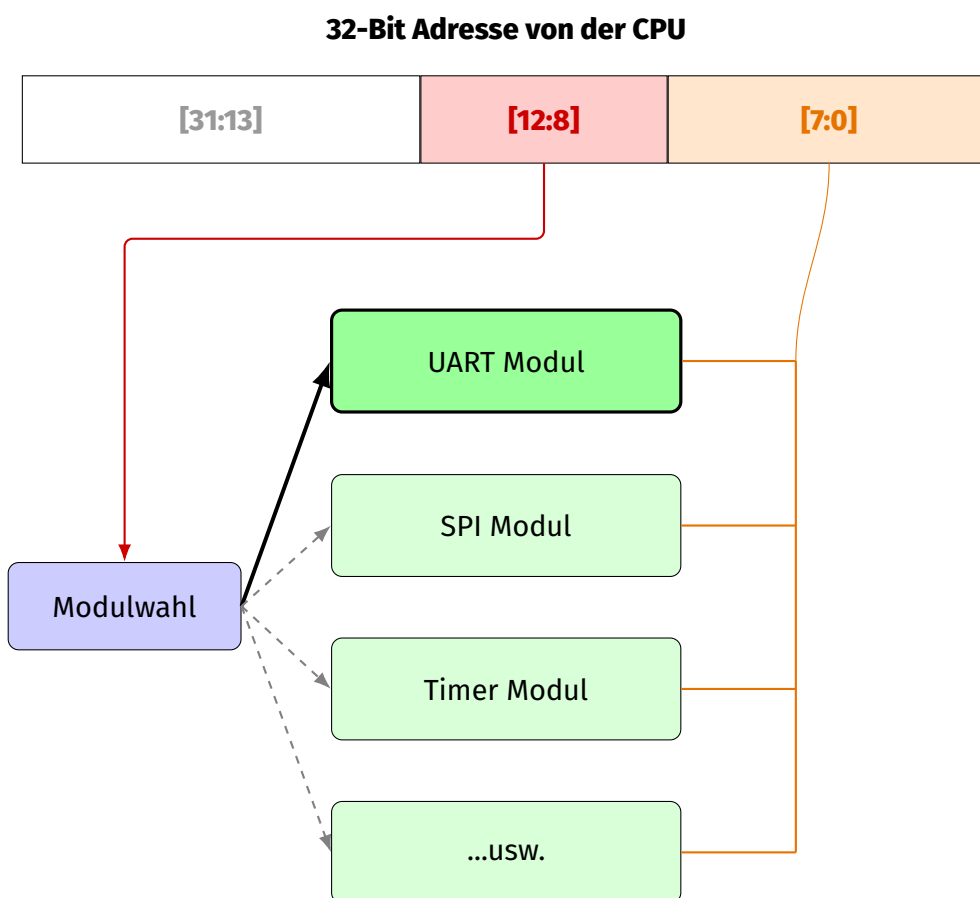


Abbildung 2.2: Prinzip der Adressdekodierung: Die Bits `[12:8]` steuern die *Modulwahl* (links). Die Bits `[7:0]` werden als *Register-Offset* an alle Module verteilt (rechts), aber nur vom ausgewählten Modul (hier: UART) verwendet.

2.4.2 Datenfluss und Steuersignale

Bei Zugriffen im Peripheriebereich nimmt der Bus die Adresse, Daten (bei Schreibvorgängen) und Steuersignale entgegen:

- **Schreibvorgang:** Die CPU sendet Daten und das Write-Enable-Signal an das ausgewählte Register.
- **Lesevorgang:** Das Peripheriemodul gibt die angeforderten Daten zurück zur CPU.

Durch diesen Bus kann Peripherie einfach angebunden und das System übersichtlich erweitert werden. Mehr Details zu den einzelnen Modulen folgen in Kapitel [3](#).

Kapitel 3

Hardwareimplementierung in Verilog

Die Hardware von PROE_WGR-V, also der CPU-Kern und die Peripheriemodule, ist komplett in Verilog beschrieben. In diesem Kapitel werden Aufbau und Funktion der wichtigsten Verilog-Module vorgestellt. Der Quellcode liegt im Verzeichnis `/rtl` und seinen Unterordnern. Auch auf spezielle Anpassungen für FPGAs im `/WGR-V-MAX` Verzeichnis wird kurz eingegangen.

3.1 Projektstruktur `/rtl`

Das Verzeichnis `/rtl` enthält den Kern des Projekts: den RISC-V CPU-Kern und die wichtigsten Basismodule. Eine ausführliche, automatisch generierte Dokumentation der Module ist online unter [github.btflv.de/PROE_WGR-V/verilog/](https://github.com/btflv.de/PROE_WGR-V/verilog/) zu finden. Im Folgenden werden die zentralen Module kurz beschrieben.

3.1.1 CPU-Kern (`cpu.v`)

Das Modul `cpu.v` ist der Hauptprozessor. Es unterstützt das RV32I Instruktionsset und die kleinere RV32E-Variante. Die wichtigsten Instruktionsfelder wie Opcodes, Funct3 und Funct7 sind als `localparam` direkt im Code abgelegt (z.B. `localparam F3_BLTU = 3'b110;`), was das Verständnis erleichtert.

Die CPU kann auf langsame Speicher oder Peripherie warten – dazu wird das `mem_busy` Signal genutzt: Ist dieses Signal aktiv, pausiert die CPU, bis der Zugriff abgeschlossen ist. Die Architektur ist einfach gehalten und kommt entweder ganz ohne Pipeline oder nur mit einer sehr einfachen Pipeline aus, um die Komplexität gering zu halten.

3.1.2 ALU (`alu.v`)

Die ALU (`alu.v`) führt alle arithmetischen und logischen Befehle des RV32I/E-Satzes aus. Dazu zählen Addition, Subtraktion, logische Verknüpfungen (UND, ODER, XOR), Shift-

Operationen und Vergleiche (Set-Less-Than). Die ALU arbeitet mit Operanden aus der Registerdatei oder mit Werten, die direkt in der Instruktion stehen. Gesteuert wird sie von der Steuereinheit der CPU.

3.1.3 Register File (`register_file.v`)

Das Modul `register_file.v` stellt die Registerdatei bereit. Sie ist 32 Bit breit und kann je nach Modus 32 Register (RV32I) oder 16 Register (RV32E) bereitstellen. Die Umstellung ist einfach möglich und spart Platz bei kleinen Zielplattformen. Die Registerdatei hat zwei Leseports und einen Schreibport, was parallele Zugriffe in einem Takt ermöglicht.

3.1.4 Top-Level Modul (`wgr_v_max.v`)

Das Modul `wgr_v_max.v` ist das oberste Modul des FPGA-Designs. Hier werden CPU-Kern (`cpu.v`), RAM (meist als FPGA-IP-Kern) und der Peripheriebus (`peripheral_bus.v`) miteinander verbunden. Für Simulation und Debugging können interne Signale (z.B. `write_data[31:0]`) nach außen geführt werden. Für die FPGA-Synthese müssen diese entweder entfernt oder als „Virtual Pins“ in Quartus deklariert werden, um Fehler zu vermeiden. Die Systemtaktfrequenz wird über `CLK_FREQ` in `defines.v` eingestellt und muss zur realen Hardware passen.

3.2 Peripheriemodule (`/rtl/peripherals`)

Im Ordner `/rtl/peripherals` liegen die Verilog-Module für die verschiedenen Peripheriemodule, die dem Prozessor zusätzliche Funktionen geben.

3.2.1 Peripheral Bus (`peripheral_bus.v`)

Das Modul `peripheral_bus.v` ist die zentrale Schnittstelle zwischen CPU und Peripheriemodulen. Es übernimmt die Adressdekodierung: `address[12:8]` wählt das Modul aus, `address[7:0]` bestimmt das Register innerhalb des Moduls. Lese- und Schreibsignale sowie Daten werden entsprechend weitergeleitet.

3.2.2 UART (`uart.v`)

Das UART-Modul (`uart.v`) bietet eine serielle Schnittstelle mit separaten FIFO-Buffern für Senden (TX) und Empfangen (RX). Damit kann die CPU größere Datenmengen am Stück übertragen, ohne jedes einzelne Byte einzeln behandeln zu müssen. Unterstützt werden gängige Baudraten von 300 bis 115200 Baud, die per Register einstellbar sind.

3.2.3 SPI (spi.v)

Das SPI-Modul (spi.v) stellt eine SPI-Schnittstelle bereit, ebenfalls mit FIFO-Puffern für Sende- und Empfangsdaten. Der Chip Select (CS) kann automatisch oder manuell gesteuert werden. Die SPI-Frequenz wird über einen Teiler eingestellt und kann bis zur halben Systemtaktfrequenz betragen.

3.2.4 PWM Timer (pwm_timer.v)

Das Modul pwm_timer.v ist ein 32-Bit-Timer, der auch ein PWM-Signal erzeugen kann. Die Frequenz kann über einen Taktteiler angepasst werden, Periode und Duty-Cycle sind per Register einstellbar. Es gibt auch einen einfachen 50%-Modus.

3.2.5 Debug Module (debug_module.v)

Dieses Modul ist vor allem für Simulation und Debugging gedacht. Es stellt ein einfaches, les- und schreibbares Register bereit, das von der CPU und der Testbench genutzt werden kann, um Werte auszutauschen oder gezielt das Verhalten der CPU zu testen.

3.2.6 System Timer (system_timer.v)

Der system_timer.v bietet Zähler für Milli- und Mikrosekunden, die von der Software über Speicherzugriffe ausgelesen oder zurückgesetzt werden können. Damit lassen sich Zeitmessungen und Delays einfach realisieren.

3.2.7 FIFO (fifo.v)

Das Modul fifo.v ist eine generische FIFO-Struktur, wie sie bei UART und SPI zum Puffern von Daten eingesetzt wird. Die Breite und Tiefe der FIFO lassen sich anpassen.

3.2.8 GPIO (gpio.v)

Das GPIO-Modul ermöglicht digitale Ein- und Ausgabe über mehrere Pins. Es nutzt getrennte Signale für Eingabe, Ausgabe und Richtungssteuerung, um mit bestimmten ASIC-Designflows kompatibel zu sein. Für reine FPGA-Projekte könnten im Top-Modul auch inout-Pins verwendet werden.

3.2.9 WS2812B Controller (ws2812b.v)

Dieses Modul steuert adressierbare WS2812B-RGB-LEDs an und erzeugt die speziellen Timings, die für das WS2812B-Protokoll notwendig sind.

3.2.10 Sequentieller Dividierer (seq_divider.v)

Das Modul `seq_divider.v` stellt eine Hardwarelösung für die 32-Bit-Ganzzahldivision bereit. Die Division wird schrittweise (sequentiell) durchgeführt und benötigt daher mehrere Taktzyklen, bis das Ergebnis bereitsteht.

3.2.11 Sequentieller Multiplizierer (seq_multiplier.v)

Das Modul `seq_multiplier.v` realisiert eine 32-Bit-Ganzzahlmultiplikation in Hardware. Auch hier erfolgt die Berechnung sequentiell über mehrere Taktzyklen, was die Implementierung einfach hält und Ressourcen spart.

3.3 FPGA Implementierung (/WGR-V-MAX)

Das Verzeichnis `/WGR-V-MAX` enthält alle nötigen Dateien und die Struktur, um das WGR-V System auf einem Intel/Altera FPGA (z.B. mit Quartus Prime) zu implementieren.

- **IP-Verzeichnis:** Hier liegen die IP-Kerne für die FPGA-Version, besonders der RAM-Baustein (`altsyncram` oder ähnlich). Im Beispiel ist der RAM auf 8192 Wörter zu 32 Bit eingestellt.
- **mem-Verzeichnis:** Enthält die Datei `wgr_flat.hex`, mit der der FPGA-RAM beim Start mit Programm und Daten geladen wird.
- **tb_sim-Verzeichnis:** Beinhaltet die Testbench (`wgr_v_max_tb.v`) für Simulationen, etwa mit QuestaSim.

Die von der RISC-V Toolchain erzeugten Hex-Dateien sind nicht direkt mit Quartus kompatibel. Das Python-Skript `scripts/hex_conv/conv_hex.py` wandelt sie in das passende Format um, berücksichtigt dabei einen Adress-Offset und füllt den Speicher bis zur gewünschten Größe auf.

Kapitel 4

ASIC Implementierung

Neben der FPGA-Version wurde auch eine ASIC (Application-Specific Integrated Circuit) Implementierung des WGR-V Prozessors realisiert. Ziel war es, den Prozessor als echten Chip umzusetzen und den gesamten Ablauf vom Verilog-Code bis zum fertigen Layout kennenzulernen. Wegen der spezialisierten Werkzeuge und Skripte gibt es für die ASIC-Entwicklung ein eigenes Repository.

4.1 Übersicht und Repository

Die Entwicklung der ASIC-Version erfolgt in einem eigenen öffentlichen GitHub-Repository:

https://github.com/BTFLV/BTFLV-PROE_WGR-V-ASIC

Die Trennung hilft dabei, die für ASIC nötigen Automatisierungen (z.B. CI/CD-Skripte für Synthese und Tests) von der FPGA-Entwicklung klar zu unterscheiden.

4.2 Tools und Prozess

Für die ASIC-Implementierung des WGR-V wurden verschiedene Open-Source EDA-Tools und ein spezielles Technologie-Kit (PDK) verwendet:

- **Synthese:** Der Weg vom Verilog-Code bis zum finalen Chip-Layout (GDSII) erfolgt mit **OpenLane2**. Dieses Tool steuert alle wichtigen Schritte, darunter Synthese (z.B. mit Yosys), Platzierung, Routing und Timing-Analyse, und nutzt dafür mehrere Open-Source-Programme im Hintergrund.
- **Simulation:** Zur Überprüfung des Designs werden **Verilator** (für schnelle Simulationen) und **cocotb** (für flexible, Python-basierte Testbenches) eingesetzt.

- **Technologie:** Als Fertigungsprozess wird das offene **SkyWater 130nm PDK** (SKY130) verwendet, das einen Einstieg in die reale Chipentwicklung mit modernen Werkzeugen ermöglicht.

4.3 CI/CD Workflow im ASIC Repository

Das ASIC-Repository verwendet automatisierte Skripte (CI/CD), um bei jeder Änderung den gesamten Chip-Flow durchzuführen und zu testen:

- **Automatische Synthese:** Jede Codeänderung startet automatisch den OpenLane2-Flow. Dazu gehören Code-Checks, Synthese, Platzierung, Routing, Timing-Analyse und die Erstellung des finalen Layouts.
- **Automatische Tests:** Nach der Synthese laufen Simulationen mit Verilator und coctb, um das Design sowohl vor als auch nach der Synthese zu überprüfen.
- **Berichte und Ergebnisse:** Die Pipeline erstellt Berichte zu Synthese, Timing, Fläche und Testergebnissen. Wichtige Dateien wie das GDSII-Layout und Simulationsprotokolle werden gespeichert und bereitgestellt.
- **Schnelle Iteration:** Durch die Automatisierung können Designänderungen schnell geprüft werden, was den Entwicklungsprozess deutlich beschleunigt und Fehler frühzeitig erkennt.

4.4 Speicheranbindung im ASIC

Im Unterschied zur FPGA-Version mit internem Block-RAM nutzt die ASIC-Version des WGR-V einen externen **SPI-FRAM** (Ferroelectric RAM) als gemeinsamen Programm- und Datenspeicher. FRAM ist nichtflüchtig (wie Flash), lässt sich aber ähnlich schnell beschreiben wie SRAM.

- Der SPI-FRAM wird über die gleichen Pins angebunden, die in der FPGA-Version für das Standard-SPI-Modul (`spi.v`) vorgesehen sind. Im ASIC-Design kommt dafür eine speziell angepasste SPI-Schnittstelle zum Einsatz.
- Das Modul `fram_ram.v` verbindet das Speicherinterface der CPU mit dem SPI-Controller (`fram_spi.v`) für den FRAM. Für die CPU wirkt der Speicher wie ein normaler RAM, auch wenn intern SPI-Kommunikation abläuft.
- Das Modul `mb85rs64v.v` dient als Verilog-Modell eines typischen FRAM-Bausteins (z.B. MB85RS64V mit 64 Kbit Kapazität) für die Simulation.

So kann im ASIC ein nichtflüchtiger Speicher ohne aufwändige On-Chip-Flash-Technik genutzt werden.

4.5 Chip-Layout

Als Ergebnis des ASIC-Designflows mit OpenLane2 entsteht das physische Layout des Chips.

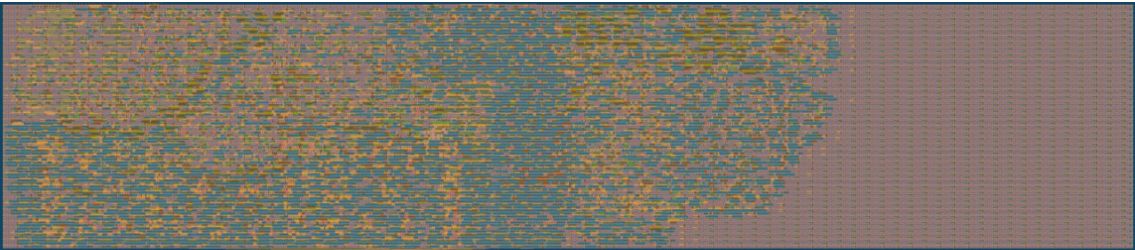


Abbildung 4.1: Gerenderte Darstellung des WGR-V ASIC Layouts.

Kenndaten des Chips:

- **Gesamtfläche:** 0,192 mm²
- **Kernauslastung:** ca. 41 %
- **Anzahl der Transistoren:** ca. 80.000

Diese Werte zeigen den Umfang und die Komplexität des Designs im SkyWater 130nm Prozess.

4.6 Enthaltene Peripherie im ASIC

Um die Chipfläche und Komplexität zu begrenzen, enthält das erste ASIC-Design nur eine Auswahl der Peripheriemodule. Tabelle 4.1 zeigt, welche Module integriert sind und welche weggelassen wurden.

Tabelle 4.1: Status der Peripheriemodule im WGR-V ASIC Design

Peripherie-Modul	Im ASIC enthalten
debug_module	Ja
uart	Ja
pwm_timer	Ja
system_timer	Ja
spi (generisch)	Nein (durch FRAM-SPI ersetzt)
gpio	Nein
ws2812b	Nein
seq_multiplier	Nein
seq_divider	Nein

Für die ASIC-Version wurde also ein kompakter Satz von Modulen ausgewählt, um grundlegende Kommunikation (UART), Zeitmessung (Timer) und Debugging zu ermöglichen.

Kapitel 5

Software und Firmware (/src)

Zum Prozessorsystem gehört auch passende Software, die den Prozessor nutzbar macht und seine Funktionalität testet. Im Verzeichnis `/src` ist der gesamte Quellcode für den WGR-V Prozessor gesammelt: Bibliotheken für den Hardwarezugriff (HAL), Linker-Skripte, Startcode, Beispielprogramme in C und Assembler sowie verschiedene Testprogramme.

5.1 Übersicht und Dokumentation

Das Hauptverzeichnis `/src` und seine Unterordner enthalten den kompletten C- und Assembler-Code zur Ansteuerung des WGR-V Prozessors und seiner Peripherie. Für die HAL-Bibliotheken gibt es eine automatisch generierte Dokumentation unter [github.btflv.de/PROE_WGR-V/c/](https://github.com/btflv.de/PROE_WGR-V/c/) erstellt mit Doxygen. Die Dokumentation bietet eine Übersicht aller Funktionen und Datentypen.

5.2 Hardware Abstraction Layer (HAL)

Die Hardware Abstraction Layer (HAL) im Verzeichnis `src/lib/` bildet das Bindeglied zwischen Hardware und Anwendungscode. Sie sorgt dafür, dass der Zugriff auf die Hardware einfacher und übersichtlicher wird und erleichtert die Portierung der Software auf verschiedene Ausprägungen des WGR-V Systems (zum Beispiel FPGA oder ASIC mit unterschiedlichen Peripheriemodulen).

Die HAL gliedert sich in mehrere zentrale Dateien, die jeweils unterschiedliche Aufgaben übernehmen:

- **wgrtypes.h**: Diese Datei definiert die grundlegenden Datentypen für das gesamte Projekt, wie `uint8_t`, `int32_t`, `bool` sowie spezielle Enumerationen (z.B. für UART-Baudraten). So ist sichergestellt, dass der Code auf unterschiedlichen Plattformen und mit verschiedenen Compilern konsistent bleibt.

- **wgrhal.h** und **wgrhal.c**: Sie bilden das Herzstück der HAL für die Basis-Peripherie. Hier sind Funktionen für Standardaufgaben wie Speicheroperationen (`memcpy`, `memset`), String-Bearbeitung (`strlen`, `strcmp`), sowie Debug-Ausgaben enthalten. Über UART-Funktionen (`uart_enable`, `uart_set_baud`, `uart_write_byte`, `uart_print`) kann die serielle Schnittstelle direkt genutzt werden. Außerdem gibt es Timer-Funktionen (`millis`, `micros`, `delay`), PWM-Steuerung (`pwm_set_period`, `pwm_set_duty`) und GPIO-Zugriffe (`gpio_write_pin`, `gpio_read_pin`). Das Makro `HWREG32(addr)` erlaubt den direkten Zugriff auf 32-Bit-Hardware-Register. In `wgrhal.h` werden außerdem die Basisadressen der Peripheriemodule und die Offsets für die einzelnen Register definiert, z.B. `UART_BASE_ADDR` und `TIME_BASE_ADDR`. Über die Defines `MALLOC`, `PWM_NOTES` und `SSD1351` können bei Bedarf zusätzliche Funktionen zugeschaltet werden.
- **wgrhal_ext.h** und **wgrhal_ext.c**: Diese Dateien erweitern die HAL um spezielle Features, etwa:
 - Zugriff auf Hardware-Multiplizierer und -Dividierer mit Funktionen wie `mult_calc()` und `div_calc_quotient()`, die direkt auf die entsprechenden Hardware-Einheiten zugreifen.
 - Umfassende SPI-Kommunikationsfunktionen (`spi_enable`, `spi_set_clock_divider`, `spi_write_byte` usw.).
 - Ansteuerung von WS2812B-LEDs (`ws2812_set_color`, `ws2812_fill`) über das dazugehörige Hardwaremodul.
 - PWM-basierte Tonerzeugung mit `pwm_play_note()`, sofern `PWM_NOTES` aktiviert ist. Hierzu gibt es eine Frequenztabelle für verschiedene Noten.
 - Dynamische Speicherverwaltung per `malloc()`, `free()`, `realloc()`, `calloc()`, sofern `MALLOC` aktiviert ist.
 - Unterstützung für SSD1351 OLED-Displays, inklusive Initialisierung, Text- und Grafikausgabe, wenn das entsprechende Makro definiert ist.
- **wgrlib.c**: Diese Datei stellt die sogenannten Compiler-Runtime-Funktionen bereit, die der GCC-Compiler manchmal benötigt, wenn keine passende Hardware für bestimmte Operationen vorhanden ist. Dazu gehören Software-Implementierungen für:
 - Gleitkommaarithmetik (`__floatsisf`, `__fixsfsi`, `__addsf3`, `__subsf3`, `__mulsf3`, `__divsf3`).
 - Komplexere Ganzzahloperationen für 32- und 64-Bit (`__mulsi3`, `__udivsi3`, `__divsi3`, `__modsi3`, `__muldi3`, `__udivdi3`, `__divdi3`, `__moddi3`). Die

32-Bit-Versionen können, falls das Makro `HARDWAREDIVISION` aktiv ist, auch direkt auf die Hardware zugreifen.

- Bitmanipulationsfunktionen wie `__clzsi2` (Anzahl führender Nullen zählen).

Diese Funktionen werden vom Compiler automatisch eingebunden, wenn ein entsprechender C-Code verwendet wird, für den keine direkte Maschineninstruktion existiert.

5.3 Linker und Startup-Code

Im Verzeichnis `src/linker/` befinden sich wichtige Dateien, die den Start und die Speicheraufteilung der Software auf dem WGR-V Prozessor steuern.

- **crt0.s**: Dieser Assembler-Code bildet den C-Startup, also das allererste Programmstück, das nach einem Reset ausgeführt wird. Er bereitet das System für die Ausführung von C-Programmen vor. Zu den wichtigsten Aufgaben gehören:
 1. Initialisieren des Stack Pointers (`sp`), meist auf das Ende des reservierten RAMs (`_estack`), das durch das Linker-Skript festgelegt wird.
 2. Starten der `main`-Funktion durch einen Sprungbefehl (`jal ra, main`).
 3. Eine Endlosschleife am Ende, falls `main` jemals zurückkehrt.
- **Linker-Skripte** (normalerweise mit der Endung `.ld`): Diese Skripte geben dem Linker vor, wie Programmteile wie Code (`.text`), initialisierte Daten (`.data`), uninitialisierte Daten (`.bss`), Stack und Heap im Speicher verteilt werden. Außerdem werden wichtige Symbole wie `_estack` (für den Stack) oder `_heap_start` und `_heap_end` (für die Speicherverwaltung) definiert, die von der Software verwendet werden.

5.4 Beispielprojekte

Zur Demonstration der Nutzung des WGR-V Prozessors und seiner HAL gibt es mehrere Beispielprojekte im Verzeichnis `src/project/` (für C) und `src/project_asm/` (für Assembler).

- **C-Beispielprojekt (src/project/main.c)**: Hier wird eine interaktive Terminalanwendung umgesetzt, die über die serielle Schnittstelle (UART) und das SSD1351 OLED-Display bedient werden kann.
 - In der `setup()`-Funktion werden Peripherie wie SPI (für das Display), das Terminal und die PWM-Notenberechnung initialisiert sowie ein Textpuffer mit `malloc()` reserviert.

- Die `read_cmd()`-Funktion liest Benutzerbefehle ein, die dann von `interpret_cmd()` ausgewertet und umgesetzt werden.
- Unterstützt werden unter anderem folgende Kommandos:
 - * `mult <a> <b>`: Hardware-Multiplikation
 - * `div <a> <b>`: Hardware-Division
 - * `note <note> <oct>`: PWM-basierte Tonausgabe
 - * `ws <r> <g> <b> [led]`: Steuerung der WS2812B LEDs
 - * `pin [0 .. 5] -on/-off`: GPIO-Pins schalten
 - * `time [-ms]`: Anzeige der Systemzeit
 - * `heap`: Anzeige des freien Heaps
 - * `clear, invert, help`: Steuerbefehle für das Terminal

Dieses Beispiel nutzt viele der HAL-Funktionen und eignet sich gut zum Einstieg.

- **Assembler-Beispiel (`src/project_asm/main_asm.S`)**: Ein in RISC-V Assembler geschriebenes Beispiel, das die Fibonacci-Folge berechnet. Die beiden Startwerte werden auf dem Stack abgelegt, in jeder Runde werden die obersten Werte addiert und das Ergebnis sowie der alte Wert zurückgeschrieben. Der jeweils aktuelle Fibonacci-Wert wird zusätzlich ins Debug-Register (`0x00000100`) geschrieben, damit die Berechnung von außen beobachtet werden kann (z.B. in einer Simulation).

5.5 Funktionale Tests (/src/tests)

Zur Überprüfung des WGR-V Prozessors und seiner Peripherie gibt es im Verzeichnis `src/tests/` verschiedene Testprogramme. Sie prüfen wichtige Aspekte der Hardware und Software auf Korrektheit.

- **`benchmark/main.c`**: Dieses Programm führt verschiedene Benchmarks durch, um die Leistungsfähigkeit des Prozessors bei unterschiedlichen Operationen zu messen. Es testet unter anderem:
 - Einfache Schleifen (`benchmark_loop`).
 - Array-Schreib- und Lesezugriffe (`benchmark_array_write`, `benchmark_array_read`).
 - Bedingte Anweisungen (`benchmark_conditional`).
 - Bitweise Operationen (XOR, 8-Bit komplex, 16-Bit Shift).
 - Einfache Logik und Inkrementoperationen.
 - Dynamische Speicherallokation (`benchmark_malloc_free`).
 - Arithmetik mit kleineren Datentypen (8-Bit und 16-Bit Additionen).

- Gleitkommaoperationen (Addition, Multiplikation, Division) – diese nutzen die Software-Routinen aus `wgrlib.c`.
- Integer-Multiplikation und -Division.
- Bitweise Shifts und Typkonvertierungen (Integer zu Float und zurück).

Die Ergebnisse (Laufzeiten) werden über die Debug-Schnittstelle ausgegeben.

- **general/main.c:** Dieses Testprogramm prüft eine breite Palette von grundlegenden CPU- und Systemfunktionen:

- Arithmetische Grenzfälle (z.B. Überläufe bei Addition/Subtraktion, Division durch -1).
- Logische und arithmetische Shift-Operationen mit negativen Zahlen.
- Vergleiche zwischen vorzeichenbehafteten und vorzeichenlosen Zahlen.
- Speicherzugriffe auf nicht-alignierte Adressen (führt je nach CPU-Implementierung zu Exceptions oder undefiniertem Verhalten; hier wird das Ergebnis von Lesezugriffen auf `misaligned[1]` als `uint16_t` und `uint32_t` geprüft).
- Ein Test für selbstmodifizierenden Code (überschreibt eine Instruktion im Speicher und führt sie danach ggf. aus; dies ist mit Vorsicht zu genießen und hängt stark von der Speicherarchitektur und Caching-Mechanismen ab).
- Tiefe Rekursion (`deep_recursion`) zur Überprüfung des Stack-Handlings.
- Einfacher Test zur Branch Prediction (zählt in einer Schleife abwechselnd hoch und runter).
- Emulierte 64-Bit Multiplikation.
- Grundlegende Gleitkommaoperationen, inklusive Division durch Null (Erzeugung von Inf) und 0.0/0.0 (Erzeugung von NaN).

Die Ergebnisse werden ebenfalls via `debug_write` ausgegeben.

- **hardware_mult_div/main.c:** Dieses Programm testet und benchmarkt spezifisch die Hardware-Multiplizierer und -Dividierer, falls vorhanden und über die HAL ansprechbar.

- Es verwendet einen Xorshift-Algorithmus (`xorshift32`, `xorshift64`) zur Generierung von Zufallszahlen für die Testdaten.
- Es misst die Ausführungszeit für eine Reihe von 32-Bit-Multiplikationen, 32-Bit-Divisionen und 64-Bit-Multiplikationen (die Ergebnisse werden auf 32 Bit reduziert).

- Im Quellcode sind kommentierte Ergebnisse für eine Software- und Hardware-Implementierung bei 10MHz enthalten, die einen deutlichen Geschwindigkeitsvorteil der Hardware zeigen.
- **sh_sb_lh_lb_asm/main_asm.S:** Dieses Testprogramm ist in RISC-V Assembler geschrieben und fokussiert sich auf die korrekte Funktionsweise der Load- und Store-Instruktionen für Bytes (8 Bit) und Halfwords (16 Bit).
 - Es werden positive und negative Werte verwendet, um die Vorzeichenerweiterung bei Load-Instruktionen zu überprüfen.
 - Getestete Instruktionen: sh (Store Halfword), lh (Load Halfword, sign-extended), lhu (Load Halfword, zero-extended), sb (Store Byte), lb (Load Byte, sign-extended), lbu (Load Byte, zero-extended).
 - Die Operationen werden auf dem Stack ausgeführt, wobei der Stackpointer (sp) entsprechend angepasst wird.
 - Die Ergebnisse der Load-Instruktionen werden in das Register a0 geladen und anschließend in das Debug-Register (DEBUG_REG an Adresse 0x00000100) geschrieben, um sie extern verifizieren zu können.
 - Das Programm läuft in einer Endlosschleife (j loop).

Diese Testprogramme helfen, Fehler früh zu erkennen und die RISC-V-Kompatibilität abzusichern.

Kapitel 6

Entwicklungsworkflow und Werkzeuge

Verschiedene Skripte und Werkzeuge automatisieren wichtige Aufgaben – von der Kompilierung über die Speicherinitialisierung bis hin zur Simulation und Dokumentation. Das Ziel ist, wiederkehrende Entwicklungsschritte zu vereinfachen und zu beschleunigen.

6.1 Skripte zur Automatisierung (/scripts)

Im Verzeichnis /scripts liegen diverse Batch- und Python-Skripte, die für typische Aufgaben wie Software-Kompilierung, Formatumwandlung und Simulation sorgen.

6.1.1 Konfigurationsskripte

Skripte wie `parameter_setup.bat` und `path_setup.bat` dienen wahrscheinlich dazu, Umgebungsvariablen und Pfade zu wichtigen Tools (z.B. RISC-V Toolchain, Quartus, QuestaSim) einzurichten. Sie sorgen für eine einheitliche Arbeitsumgebung und erleichtern die Nutzung aller Automatisierungsskripte im Team.

6.1.2 Hex-Konvertierungsskripte

Damit der kompilierte Programmcode in den FPGA-Speicher geladen oder in Simulationen genutzt werden kann, muss er in ein bestimmtes Hex-Format umgewandelt werden. Dazu gibt es zwei Python-Skripte:

conv_hex.py

Dieses Python-Skript wandelt die von der RISC-V Toolchain erzeugten Hex-Dateien (objcopy) in ein für die RAM-IP-Kerne von Quartus (z.B. altsyncram) geeignetes Format um.

- **Wortadressierung:** Jede Zeile der Ausgabedatei enthält ein 32-Bit-Wort.
- **Adressverschiebung:** Mit dem Parameter `shift_amount` kann das Hexfile in einen höheren Speicherbereich verschoben werden, z.B. um unteren Speicher für Peripherie zu reservieren (`0x4000` im Projekt).
- **Speicherauffüllung:** Nicht belegte Speicherstellen werden mit Nullen aufgefüllt, bis zur gewünschten Tiefe (`memory_depth`).
- **EOF-Record:** Am Ende wird ein End-of-File-Eintrag (`:00000001FF`) angefügt, wie ihn manche Reader erwarten.

```
1 # Verwendung:  
2 conv_hex.py <input.hex> <output.hex> <shift_amount in hex> <memory_depth>  
3 # Beispiel:  
4 conv_hex.py wgr.hex wgr_flat.hex 0x4000 8192
```

Listing 6.1: Aufrufbeispiel für conv_hex.py

conv_fram.py

Für die ASIC-Variante mit externem SPI-FRAM ist ein anderes Format nötig, das von `$readmemh` in Verilog verstanden wird. `conv_fram.py` wandelt die von `conv_hex.py` erzeugte Datei entsprechend um.

- **Eingabe:** Erwartet eine Quartus-kompatible Hex-Datei.
- **Ausgabe:** Schreibt jedes 32-Bit-Wort als vier Hex-Bytes (MSB zuerst), getrennt durch Leerzeichen, in eine neue Zeile. Beispiel: `:04000400FF81011364` wird zu `FF 81 01 13`.
- **Speichergroße:** Die Anzahl der ausgegebenen 32-Bit-Worte kann vorgegeben werden (`memory_size`).

```
1 # Verwendung:  
2 conv_fram.py <input.hex> <output.hex> <memory_size>  
3  
4 # Beispiel:  
5 conv_fram.py wgr_flat.hex wgr_fram.hex 2048
```

Listing 6.2: Aufrufbeispiel für conv_fram.py

6.1.3 Kompilierungs- und Simulationsskripte

Zur Steuerung des Kompilier- und Simulationsprozesses kommen Batch-Dateien und Tcl-Skripte zum Einsatz. Sie erleichtern den Umgang mit der Toolchain und der Simulation.

Beispiele für Kompilierungsskripte sind:

- `scripts/RV32I_quartus/build_main.bat`: Baut C-Projekte für die FPGA-Version.
- `scripts/RV32E_fram/build_main.bat`: Baut C-Projekte für die ASIC-Version mit FRAM.

Die Skripte führen typischerweise den Compiler, den Assembler und `objcopy` aus und rufen anschließend die passenden Hex-Konverter-Skripte auf. Die erzeugten Speicherabbilder werden direkt in die vorgesehenen Verzeichnisse (z.B. `/WGR-V-MAX/mem/`) kopiert.

Für die Simulation mit QuestaSim werden im Verzeichnis `scripts/sim/questa/` Tcl-Skripte bereitgestellt:

- `run_rtl.tcl` und `run_gl.tcl`: Starten RTL- bzw. Gate-Level-Simulationen. Sie kompilieren das Design und die Testbench und starten den Simulator.
- `run_rtl_vcd.tcl` und `run_gl_vcd.tcl`: Führen Simulationen mit zusätzlicher Erzeugung von VCD-Dateien aus, die Signalverläufe speichern. Diese können später z.B. mit GtkWave ausgewertet werden. Es wird festgelegt, welche Signale mit aufgezeichnet werden.
- `changetimescale.py`: Hilfsskript zur Umwandlung der Zeitskala in VCD-Dateien, etwa von ps auf ns, um die Lesbarkeit zu verbessern.

6.2 Dokumentationsgenerierung

Die Projektdokumentation wird automatisch mit einem GitHub Workflow (`.github/workflows/doxygen.yml`) erzeugt und veröffentlicht. Dabei kommen verschiedene Tools zum Einsatz:

- **Doxygen** für die C-Dokumentation, **Graphviz** für Diagramme, **TeX Live** für die PDF-Erstellung, **Pandoc**, Python mit `markdown` und `pygments`, sowie das eigene Skript `DoxNet.py` für die Verilog-Dokumentation.
- **C-Code**: Doxygen erzeugt aus den Kommentaren im Quellcode (`/src/lib/`) HTML- und LaTeX-Dokumentation, die anschließend als PDF kompiliert wird (`wgr-v-c-docs.pdf`).

- **Verilog-Code:** Die Verilog-Module werden mit `DoxNet.py` und einer JSON-Konfiguration (`docs/DoxNet.json`) dokumentiert. Auch hier gibt es HTML- und PDF-Ausgaben (`wgr-v-verilog-docs.pdf`).
- **Veröffentlichung:**
 - Die PDF-Dateien werden als Build-Artefakte gespeichert.
 - Die HTML-Dokumentation wird auf GitHub Pages bereitgestellt und ist über eine automatisch generierte Index-Seite zugänglich.
 - Zusätzlich wird bei jedem Push ein ZIP-Archiv des Repos samt PDFs als Release veröffentlicht.

Durch diese Automatisierung bleibt die Dokumentation immer aktuell und ist jederzeit leicht zugänglich.

Kapitel 7

Anleitung zur Nutzung des Projekts

Dieses Kapitel bietet eine praktische Anleitung zur Nutzung des PROE_WGR-V Projekts. Es werden die wichtigsten Voraussetzungen, die Schritte zur Software-Kompilierung, die Erstellung von Speicherabbildern und die Durchführung von Hardware-Simulationen erklärt, um einen schnellen Einstieg zu ermöglichen.

7.1 Voraussetzungen

Vor der Arbeit mit dem Projekt müssen einige Softwaretools und Entwicklungsumgebungen eingerichtet werden. Die meisten Skripte liegen als Batch-Dateien (.bat) vor und sind für Windows ausgelegt. Für Linux wäre eine Anpassung auf Shell-Skripte möglich, dies erfordert aber zusätzliche Anpassungen.

Benötigt werden:

- **RISC-V GCC Toolchain:** Für die Übersetzung von C- und Assembler-Code in Maschinencode (gcc, as, ld, objcopy, objdump usw.).
- **Intel Quartus Prime:** Für Synthese, FPGA-Implementierung und die Erstellung von Netzlisten. Die Version sollte zur verwendeten FPGA-Familie passen (z.B. MAX 10).
- **QuestaSim oder ModelSim:** Simulator für Verilog auf RTL- und Gate-Level. Die vorhandenen Tcl-Skripte sind dafür ausgelegt.
- **Python 3:** Für die Ausführung der Hex-Konverter- und Hilfsskripte.
- **Git:** Zur Versionsverwaltung und für das Klonen des Repos.
- **Texteditor oder IDE:** Für die Bearbeitung von Quell- und Skriptdateien (z.B. VS Code, Notepad++, Sublime Text).
- **(Optional) Waveform Viewer:** Tools wie PulseView oder GtkWave, um die bei Simulationen erzeugten VCD-Dateien zu analysieren.

Wichtig: Die Pfade zu allen wichtigen Tools (v.a. RISC-V Toolchain, Quartus, QuestaSim) sollten in der Systemumgebung (PATH) stehen oder über das Skript `path_setup.bat` richtig gesetzt werden.

7.2 Kompilieren von Softwareprojekten

Das Kompilieren von C- oder Assembler-Projekten für den WGR-V Prozessor wird durch Batch-Skripte automatisiert. Diese übernehmen alle Schritte – vom Aufruf des Compilers und Assemblers bis zur Erzeugung und Konvertierung der finalen Speicherabbilder.

Es gibt verschiedene Skripte, je nach Zielplattform und Programmiersprache:

- **FPGA (Quartus, altsyncram):**

- C-Projekte: `scripts/RV32I_quartus/build_main.bat`
- Assembler-Projekte: `scripts/RV32I_quartus/build_asm.bat`

Die Skripte nutzen Linker-Skripte für RV32I und passen die Speicheradressen für die FPGA-Architektur an (typischerweise mit `shift_amount 0x4000`).

- **ASIC (FRAM):**

- C-Projekte: `scripts/RV32E_fram/build_main.bat`
- Assembler-Projekte: `scripts/RV32E_fram/build_asm.bat`

Diese sind auf RV32E (16 Register) und den externen FRAM-Speicher abgestimmt und nutzen zusätzlich das Skript `conv_fram.py`.

Typischer Build-Ablauf:

1. Kompilierung/Assemblierung der Quellen und HAL.
2. Linken zu einer ELF-Datei mit passendem Linker-Skript.
3. Umwandlung in eine Binär- oder Hex-Datei mit `objcopy`.
4. Hex-Konvertierung:
 - Für FPGA: Umwandlung ins Quartus-Format (`conv_hex.py`).
 - Für ASIC: Weitere Umwandlung ins `$readmemh`-Format (`conv_fram.py`).

Fehlermeldungen oder Hinweise werden in der Konsole ausgegeben.

7.3 Hex-Dateien manuell generieren

Die Build-Skripte übernehmen normalerweise alle Konvertierungsschritte automatisch. Es kann jedoch sinnvoll sein, die Hex-Konvertierungsskripte (`conv_hex.py` oder `conv_fram.py`) auch einzeln auszuführen, zum Beispiel wenn Binärdateien von anderen Quellen stammen oder spezielle Parameter gewünscht werden.

Konvertierung für Quartus FPGA RAM (`altsyncram`):

Mit `conv_hex.py` kann eine von `objcopy` erzeugte Hex-Datei ins passende Quartus-Format gebracht werden. Der Aufruf über die Kommandozeile:

```
1 # Verwendung:
2 python scripts/hex_conv/conv_hex.py <input.hex> <output.hex> <shift_amount in
   hex> <memory_depth>
3
4 # Beispiel:
5 python scripts/hex_conv/conv_hex.py wgr.hex wgr_flat.hex 0x4000 8192
```

Listing 7.1: Manueller Aufruf von `conv_hex.py`

- `[input.hex]`: Ursprüngliche Hex-Datei (z.B. von `objcopy`)
- `[output.hex]`: Name der zu erzeugenden Quartus-kompatiblen Hex-Datei (z.B. `wgr_flat.hex`)
- `[shift_amount in hex]`: Startadresse im RAM (z.B. `0x4000`)
- `[memory_depth]`: RAM-Größe in 32-Bit-Worten (z.B. 8192 für 32 KiB)

Konvertierung für ASIC FRAM-Speicher (`$readmemh`):

Für den externen FRAM-Speicher im ASIC-Design wird `conv_fram.py` genutzt. Es nimmt die von `conv_hex.py` erzeugte Datei als Input.

```
1 python scripts/hex_conv/conv_fram.py [input.hex] [output.hex] [memory_size]
```

Listing 7.2: Manueller Aufruf von `conv_fram.py`

- `[input.hex]`: Quartus-kompatible Hex-Datei (von `conv_hex.py`)
- `[output.hex]`: Ausgabedatei im `$readmemh`-Format (z.B. `wgr_fram.hex`)
- `[memory_size]`: Größe des Speichers in 32-Bit-Worten (z.B. 2048 für 8 KiB FRAM)

7.4 Simulation starten

Die Überprüfung der Hardware erfolgt per Simulation mit QuestaSim (oder ModelSim). Dafür gibt es das Skript `scripts/sim/questa/run.bat`, das verschiedene Simulationsmodi über Parameter auswählt.

Aufruf aus dem Hauptverzeichnis:

```
1 scripts\sim\questa\run.bat [-gl] [-vcd] [-q]
```

Listing 7.3: Aufruf von run.bat für die Simulation

- `-gl`: Gate-Level-Simulation (setzt eine zuvor mit Quartus erzeugte Netlist voraus). Ohne diesen Parameter wird eine RTL-Simulation durchgeführt.
- `-vcd`: Erstellt eine VCD-Datei (Signalverläufe), die mit Tools wie GtkWave ausgewertet werden kann. Die aufgezeichneten Signale werden in den Tcl-Skripten festgelegt. Nach der Simulation kann das Skript `changetimescale.py` genutzt werden, um die Zeitskala (z.B. von ps auf ns) zu ändern.
- `-q`: Quiet-Modus – das Skript schließt nach der Simulation sofort, praktisch für automatisierte Abläufe.

Das Skript ruft intern die passenden Tcl-Skripte für QuestaSim auf (z.B. `run_rtl.tcl` oder `run_rtl_vcd.tcl`), die folgende Schritte ausführen:

1. Vorbereitung des Arbeitsverzeichnisses `work`.
2. Kompilieren der Verilog-Dateien (CPU, Peripherie, RAM, Testbench).
3. Simulation (ggf. inkl. VCD-Erstellung).
4. Ausführen für die voreingestellte Zeitspanne (z.B. `run 3000000ns`).
5. Beenden des Simulators.

Die Konsolenausgaben der Simulation sowie Debug-Ausgaben landen in der Datei `sim_log.txt`.

7.5 Alles in einem Schritt (`build_copy_simulate.bat`)

Für den schnellen Entwicklungszyklus gibt es das Batch-Skript `build_copy_simulate.bat`. Damit können alle wichtigen Schritte – vom Kompilieren bis zur Analyse der Simulation – mit nur einem Befehl ausgeführt werden:

1. **Kompilieren der Software:** Startet automatisch das passende Build-Skript (meist für das Standard-C-Projekt/Firmware).
2. **Hex-Konvertierung und Kopieren:** Wandelt die erzeugte Binärdatei ins benötigte Hex-Format um und legt sie am richtigen Ort für die Simulation ab.
3. **RTL-Simulation:** Startet eine Simulation in QuestaSim.
4. **VCD-Generierung:** Während der Simulation wird eine VCD-Datei (Signalverläufe) erzeugt.
5. **Pulseview starten (optional):** Das Skript versucht, Pulseview zu öffnen und direkt die erzeugte VCD-Datei zu laden. Beim ersten Mal kann es sein, dass die Datei manuell ausgewählt werden muss – nachfolgende Änderungen werden meist automatisch erkannt und neu geladen.

Kapitel 8

Zusammenfassung und Ausblick

Das Projekt PROE_WGR-V hat gezeigt, dass die Entwicklung eines eigenen RISC-V-Prozessorsystems im Rahmen einer studentischen Projektarbeit möglich ist. Von der Hardwarebeschreibung in Verilog über die Software-Toolchain bis hin zum Test auf FPGA und als ASIC wurden die wichtigsten Ziele erreicht. Dieses Kapitel fasst die Ergebnisse zusammen und gibt einen Ausblick auf sinnvolle Weiterentwicklungen.

8.1 Erreichte Ziele und Ergebnisse

Die wesentlichen Ziele des Projekts wurden wie geplant umgesetzt. Im Einzelnen wurden folgende Ergebnisse erreicht:

- **RISC-V-Prozessorkern:** Entwicklung eines 32-Bit Prozessorkerns in Verilog, wahlweise mit 32 (RV32I) oder 16 Registern (RV32E). ALU und Registerfile sind modular aufgebaut.
- **Peripheriemodule:** Der Kern ist über einen Bus mit mehreren Peripheriemodulen verbunden, darunter UART, SPI, GPIO, PWM-Timer und System-Timer.
- **Flexible Speicherarchitektur:** Unterstützung von On-Chip-RAM (FPGA, Quartus altsyncram) und externem SPI-FRAM (ASIC).
- **Hardware Abstraction Layer (HAL):** Die in C geschriebene HAL vereinfacht die Softwareentwicklung und bietet Zugriff auf Peripherie, Speicher, Strings sowie erweiterte Funktionen wie Hardware-Multiplizierer/Dividierer, WS2812B-LEDs und dynamischen Speicher.
- **Automatisierter Workflow:** Verschiedene Skripte automatisieren die Kompilierung, Formatkonvertierung und Simulation.
- **Testprogramme und Benchmarks:** Umfangreiche Tests prüfen die Korrektheit der CPU und der Peripherie sowie die Leistung.

- **ASIC-Implementierung:** Der Prozessor wurde im SkyWater 130nm-Prozess mit Open-Source-EDA-Tools umgesetzt. Das Chip-Layout umfasst ca. 80.000 Transistoren.
- **Dokumentation:** Die Projektdokumentation wird automatisiert erzeugt (HTML und PDF) und steht jederzeit aktuell zur Verfügung.

8.2 Mögliche zukünftige Arbeiten

Das Projekt bildet eine gute Basis für spätere Erweiterungen und Verbesserungen. Denkbar sind z. B.:

- **Weitere Instruktionserweiterungen:** Implementierung zusätzlicher RISC-V-Extensions, z. B. Hardware-Multiplizierer/Dividierer ("MExtension) oder komprimierte Instruktionen ("CEExtension).
- **Weitere Peripheriemodule:** Zusätzliche Schnittstellen wie I2C oder CAN für mehr Anbindungsmöglichkeiten.
- **Optimierung des ASIC-Designs:** Verbesserungen bei Fläche, Takt oder Energieverbrauch sowie Integration weiterer Module.
- **Portierung auf andere FPGA-Plattformen:** Anpassung an andere Hersteller wie Xilinx oder Lattice.
- **Ein einfaches Betriebssystem:** Aufbauend auf der HAL könnte ein kleines RTOS entstehen.

Das Projekt bietet damit eine gute Ausgangsbasis für weitere studentische Arbeiten oder praktische Anwendungen.